Interrupts and Event-Driven Programming

Introduction to ARM Cortex-M Microprocessors — Chapter 9

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Four Words That Are Not Synonyms
3. Polling and Interrupts
4. The Vector Table and NVIC
5. Handling External Interrupts
6. Writing a Handler
7. Under the Hood

Why This Chapter Matters

A program that polls a button can do nothing else. It cannot update a display, read a sensor or sleep. On a battery, it drains power while it waits. Polling does not scale. Instead, hardware can interrupt the processor when something happens. We trace one event, the PC13 button press, from the pin to the line of C that answers it.

After this chapter, you can:

1. Contrast polling and interrupts, and justify each.
2. Explain the vector table and NVIC dispatch.
3. Describe the path from pin change to ISR.
4. Configure EXTI and SYSCFG.
5. Write an ISR that clears its flag and shares data safely.
6. Use priorities and masking to control preemption.

© Muním Zabidi

Table of Contents

1. Why This Matters
2. **Four Words That Are Not Synonyms**
3. Polling and Interrupts
4. The Vector Table and NVIC
5. Handling External Interrupts
6. Writing a Handler
7. Under the Hood

Four Words That Are Not Synonyms

Students often use these words interchangeably. They name different links in a causal chain.

Term	Meaning
Event	Something happened, such as a button edge on PC13.
Interrupt request	Hardware asks the processor to respond. EXTI raises it and holds it in a pending flag.
Exception	The Cortex-M mechanism that transfers execution: the NVIC accepts the request, the processor consults the vector table.
ISR (handler)	The C function that runs, for example `EXTI15_10_IRQHandler()`.

In one line

An event causes an interrupt request, which the NVIC turns into an exception, which runs an ISR. The rest of this chapter fills in each link.

© Muním Zabidi

Table of Contents

1. Why This Matters
2. Four Words That Are Not Synonyms
3. Polling and Interrupts
4. The Vector Table and NVIC
5. Handling External Interrupts
6. Writing a Handler
7. Under the Hood

Polling and Interrupts

Polling

```
while (1) {
    if (button_pressed())
        do_something();
}
```

- The processor repeatedly checks for the event
- Simple to write and understand
- Processor time is spent checking even when nothing happens
- Response time depends on how often the event is checked

Interrupts

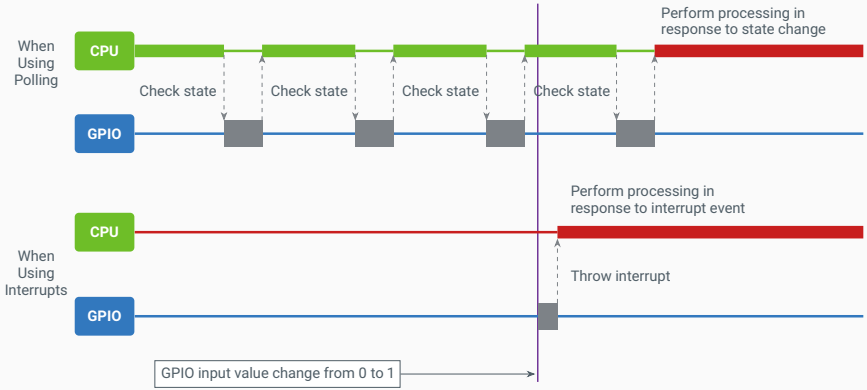
```
void EXTI15_10_IRQHandler(void) {
    do_something();
}
```

- Hardware signals the processor when the event occurs
- The processor can sleep or do other work meanwhile
- Response does not depend on continuously checking the event

The key difference

Polling asks “Has it happened yet?” repeatedly. An interrupt lets the hardware say “It has happened.”

Interrupts vs. Polling, on the Wire



- With polling, the CPU keeps checking the state, whether or not anything has changed. It only notices a change on its next check.
- With interrupts, the CPU does nothing until the GPIO line actually changes — one response, exactly when needed.

When Should I Use Each?

Both methods exist for good reasons. The real question is not which is better, but what the code needs to ask: “is this true right now?” (polling), or “tell me when this becomes true” (interrupt).

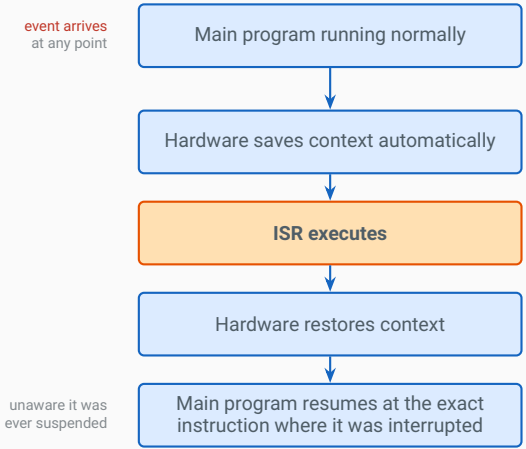
Decision factor	Favours interrupts	Favours polling
Timing of the event	Asynchronous, unpredictable	Periodic, or always true
Response deadline	Latency-sensitive	Non-critical
Power budget	Must sleep between events	Actively running anyway
Setup cost	Worth the NVIC/EXTI setup	A checking loop is enough

A button press happens at random times and needs a fast response, so interrupts win. A sensor you already read every loop pass needs nothing extra, so polling is simplest.

© Munim Zabidi

What Happens When an Interrupt Fires

We use an external interrupt: a button on a pin.



The processor saves this context for you, in hardware, before your handler runs.

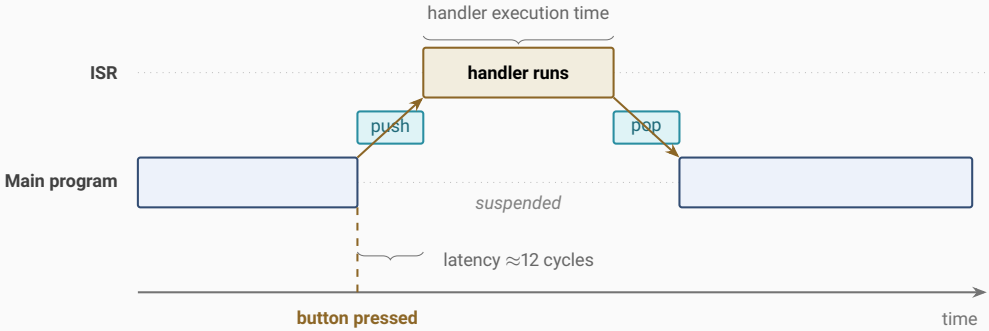
Chapter 7's stack, used automatically

It pushes `R0-R3`, `R12`, `LR`, `PC` and `xPSR` — eight words — and pops them back on return.

Your interrupted code never notices this happened.

These eight are the caller-saved registers from the AAPCS. Your handler must save anything else it uses — the C compiler does this automatically.

The Same Interrupt, Viewed Against Time



Only one row runs at any instant. The main program is paused for the whole shaded span, then resumes at the exact instruction it was about to run. Two numbers matter here: the latency before the handler starts, and the handler's own execution time — how long the main program must wait.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Four Words That Are Not Synonyms

3 Polling and Interrupts

4 **The Vector Table and NVIC**

5 Handling External Interrupts

6 Writing a Handler

7 Under the Hood

Who Routes the Event?

You have seen the event happen, and what the hardware does around it. One question is still open: how does the processor know which function to call? The answer is a chain of five links. This section, and the next, explain the middle three.



1. EXTI notices the pin change.
2. The NVIC looks at all pending interrupts and picks a winner.
3. The vector table says where that winner's code lives.
4. The processor jumps there.
5. Every interrupt uses these same five links, no matter its source.

© Muním Zabidi

The Vector Table

A table at the start of flash memory. It holds the address of every handler.

Offset	Contains
0x00	Initial SP value
0x04	Reset handler
0x08	NMI handler
0x0C	HardFault handler
...	...
0x3C	SysTick handler
0x40+	The device interrupts

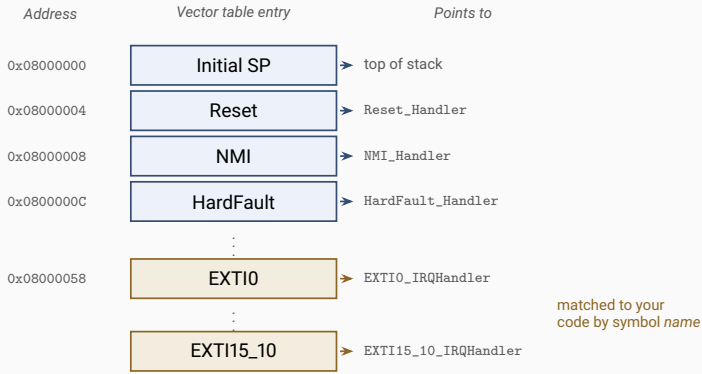
Where the initial SP comes from

The first word holds the starting stack pointer. The hardware loads this automatically at reset, before a single instruction runs.

When an interrupt fires, the processor looks up its handler here and jumps to it. This is why your handler must have exactly the right name. The startup file maps that name into the table.

© Muním Zabidi

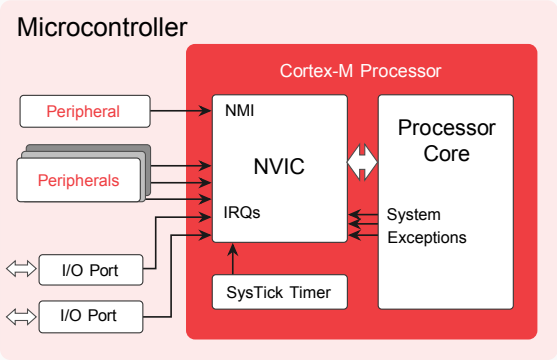
Reading the Vector Table



Two things follow from this. First, the linker matches your handler to a slot by symbol name. Misspell it, and a silent do-nothing handler runs instead of yours. Second, every entry's low bit is set to 1 to select Thumb state — this is why handler addresses look odd in a debugger.

© Muním Zabidi

Where the NVIC Sits



Every interrupt source feeds into the NVIC: external peripherals, I/O ports, the SysTick timer, even an `NMI`. The NVIC picks exactly one at a time and hands it to the Processor Core, alongside the core's own system exceptions (HardFault and the rest).

© Munim Zabidi

CMSIS function	Does
`NVIC_EnableIRQ(n)`	Enable interrupt `n`
`NVIC_DisableIRQ(n)`	Disable it
`NVIC_SetPriority(n,p)`	Set its priority

Nested means a higher-priority interrupt can interrupt a handler that is already running.

Vectored means the hardware jumps straight to the right handler. There is no software dispatch step.

Lower number means higher priority

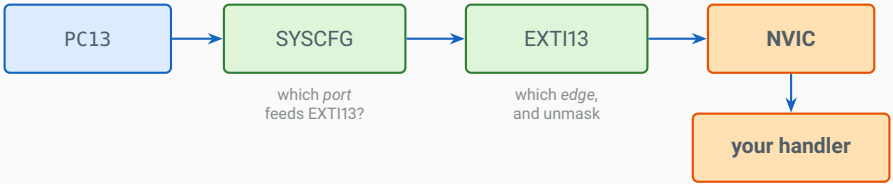
Priority `0` is the most urgent. This feels backwards at first, and it is a common source of bugs.

`__disable_irq()` and `__enable_irq()` mark a critical section: a piece of code that must not be interrupted once it starts.

Table of Contents

1. Why This Matters
2. Four Words That Are Not Synonyms
3. Polling and Interrupts
4. The Vector Table and NVIC
5. **Handling External Interrupts**
6. Writing a Handler
7. Under the Hood

From Pin to Handler

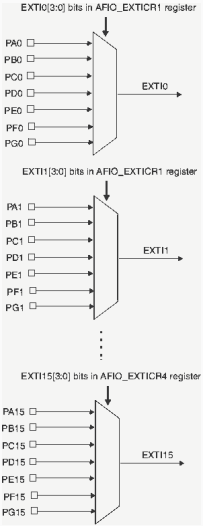


Four things to configure, in order: Route, Trigger, Enable, Handle

1. **Route** (**SYSCFG**) — choose which port's pin 13 drives EXTI13
2. **Trigger** (**EXTI**) — select the trigger edge, then unmask the line
3. **Enable** (**NVIC**) — enable the interrupt and set its priority
4. **Handle** — write the handler, with the name the vector table expects

© Muním Zabidi

EXTI Lines Are Shared



All `Px0` pins feed `EXTI0`. All `Px1` pins feed `EXTI1`, and so on. But only one pin per number can be selected at a time.

A design constraint, not a small detail

You cannot have `PA0` *and* `PB0` both generating interrupts at the same time.

Interrupt-capable inputs must use different pin numbers. This is a pin-assignment decision you make on paper, before the board is built.

`SYSCFG->EXTICR[x]` chooses which port drives each line: four bits per line, four lines per register.

The Configuration Sequence

```
1  // 0. Clocks
2  RCC->AHB1ENR |= (1U << 2);        // GPIOC
3  RCC->APB2ENR |= (1U << 14);       // SYSCFG
4
5  // 1. ROUTE: PC13 to EXTI13  (EXTICR[3], bits 7:4, value 0010 = port C)
6  SYSCFG->EXTICR[3] &= ~(0xFU << 4);
7  SYSCFG->EXTICR[3] |=  (0x2U << 4);
8
9  // 2. TRIGGER: falling edge -- an active-low button goes HIGH->LOW
10 EXTI->FTSR |= (1U << 13);
11 EXTI->IMR  |= (1U << 13);         // unmask the line
12
13 // 3. ENABLE in the NVIC
14 NVIC_SetPriority(EXTI15_10_IRQn, 2);
15 NVIC_EnableIRQ(EXTI15_10_IRQn);
16
17 // 4. HANDLE: write EXTI15_10_IRQHandler -- next section
```

Lines 10–15 all share one handler, `EXTI15_10_IRQHandler`. This is why the handler must check *which* line fired.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Four Words That Are Not Synonyms

3 Polling and Interrupts

4 The Vector Table and NVIC

5 Handling External Interrupts

6 **Writing a Handler**

7 Under the Hood

The Handler

```
volatile uint8_t button_event = 0;      // shared with main() -> volatile

void EXTI15_10_IRQHandler(void)
{
    if (EXTI->PR & (1U << 13)) {        // was it line 13?
        EXTI->PR = (1U << 13);          // clear it by writing 1
        button_event = 1;               // just record it; main() acts
    }
}
```

Three rules, and each one matters

1. The name must match the vector table exactly.
2. Clear the pending flag, by writing 1 to it. Skip this, and the handler runs forever.
3. Anything shared with `main()` must be `volatile`.

Sharing Data With an ISR

Keep handlers short

- Clear the flag
- Set a variable, or move one value
- Return

Avoid: long loops, `printf`, or anything that waits for another interrupt.

Think about latency, not lines of code.

Two hazards

- Missing `volatile`: the optimiser caches a value that the ISR is changing
- Torn reads: `main()` reads a multi-byte variable while the ISR is halfway through writing it

The pattern you just saw

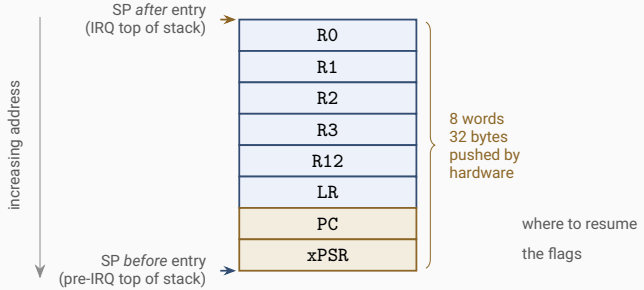
This is exactly what `button_event` did: the handler sets a flag, and `main()` does the real work (toggling the LED) the next time it checks. For anything wider than a byte, briefly disable interrupts while reading it — or use a single `uint32_t`, which the processor always reads atomically.

Table of Contents

1. Why This Matters
2. Four Words That Are Not Synonyms
3. Polling and Interrupts
4. The Vector Table and NVIC
5. Handling External Interrupts
6. Writing a Handler
7. **Under the Hood**

The Exception Stack Frame

The “save context, run handler, restore context” step is just an ordinary push and pop. Hardware does it, not an instruction you wrote. Eight words go onto the stack on entry, and the same eight come off on return.



- PC and xPSR make resuming possible: one says where to continue, the other restores the flags
- Your interrupted code never notices, and needs no special setup code of its own

You Already Know This: Priority Encoders

- Each interrupt request line is an edge-triggered flip-flop: a peripheral event sets it, and clearing the pending bit resets it.
- The NVIC examines these pending requests using priority-encoding logic—the same priority encoder used to resolve simultaneous requests.
- The winning index selects the corresponding entry in the vector table.

Why a forgotten pending-flag clear hangs the system

If your code never resets that flip-flop, the request remains asserted. When the handler returns, the NVIC sees that the line is still pending and immediately invokes the handler again. The result is an interrupt handler that runs forever—one of the most common interrupt bugs. This is why the handler you already wrote contains `EXTI->PR = (1U << 13);`.

You will rarely write these directly, since CMSIS wraps them. But it is worth knowing them, since they live at `0xE000E100`:

Register	Does
`NVIC_ISER[x]`	Set-enable: writing 1 enables that interrupt; writing 0 does nothing
`NVIC_ICER[x]`	Clear-enable: writing 1 disables
`NVIC_IPR[x]`	Priority for each channel

Separate set and clear registers avoid a read-modify-write race. This is the same design idea as GPIO's `BSRR`.

© Muním Zabidi

Sleeping Between Events

Once an event is interrupt-driven, the main loop often has nothing to do until the next one arrives. `WFI` (“wait for interrupt”) lets the CPU stop instead of spinning:

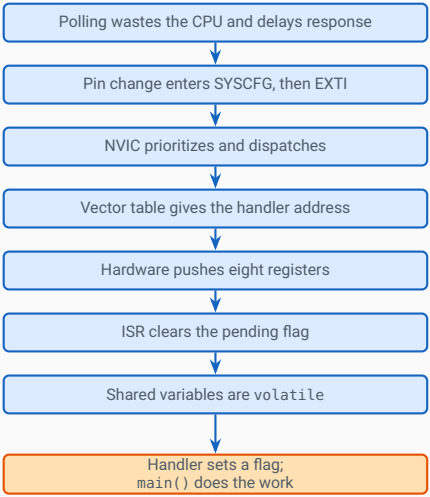
```
while (1) {
    __WFI();   // sleep until an interrupt wakes the CPU
}
```

- The core’s clock halts; the NVIC keeps watching for a pending interrupt
- On the next interrupt, the core wakes and the usual exception entry sequence runs, exactly as if it had been running
- **Interrupts** remove the need to poll for an event; **`WFI`** removes the need to keep running while waiting for one

A battery-powered sensor node can sleep almost all the time and wake only when a reading is due.

© Muním Zabidi

Summary



Terminology covered

- Polling vs interrupt
- Vector table, NVIC
- SYSCFG, EXTI
- ISR, pending flag
- Priority, preemption

Next: Chapter 10

Timers, Timing and PWM

`delay_ms()` is wrong in three ways. We time things properly.

© Muním Zabidi